GitHub Copilot Dev Days

GitHub Copilot: Seu Companheiro de IA para Cada Fluxo de Trabalho

Leonardo Tórtoro Pereira

DEMAC/IGCE/Unesp - Rio Claro

Sumário

O que é o GitHub Copilot?

Configurando VSCode

Wordle Pokémon com GitHub Copilot

Considerações finais

Section 1

O que é o GitHub Copilot?

Breve histórico do GitHub Copilot

- **1ª Onda (2021–2023):** Completamento de código
- **2ª Onda (2024–Presente):** SDLC Agentivo (chat, agent mode, planejamento)
- **3ª Onda (2026+):** Times híbridos (IA + humanos colaborando)

Evolução de um "autocompletar"para um parceiro de IA em todo o ciclo de desenvolvimento.

Modelos de IA integrados

O GitHub Copilot dá acesso a múltiplos modelos de última geração:

Anthropic

- Claude Haiku 4.5
- Claude Sonnet 4.6
- Claude Opus 4.6

OpenAI

- GPT 5-mini
- GPT 5.2
- GPT 5.3-Codex

Google

- Gemini 3.1 Pro
- Gemini 3 Flash

Você pode alternar entre modelos conforme a tarefa (performance, latência, custo).

Disponível em todos os lugares que você programa

IDEs e editores

- VS Code
- Visual Studio
- JetBrains IDEs (IntelliJ, PyCharm, etc.)
- Eclipse, Vim, Xcode

Plataformas GitHub

- GitHub.com (web)
- GitHub Mobile

Ferramentas de comunicação e projeto

- Terminal (Copilot CLI)
- Slack & Teams
- Azure DevOps Boards, Linear, Jira (novidade)

Uma única assinatura funciona em todos esses ambientes.

Section 2

Configurando VSCode

Passo 1: Instalar a extensão do GitHub Copilot

1. Abra o **VS Code**
2. Acesse a aba de **Extensões** (ícone de quadrado ou *Ctrl+Shift+X*)
3. Pesquise por *GitHub Copilot*
4. Clique em **Instalar**

*A extensão oficial é da GitHub. Recomenda-se também instalar o **GitHub Copilot Chat** (já incluso nas versões mais recentes).*

A extensão

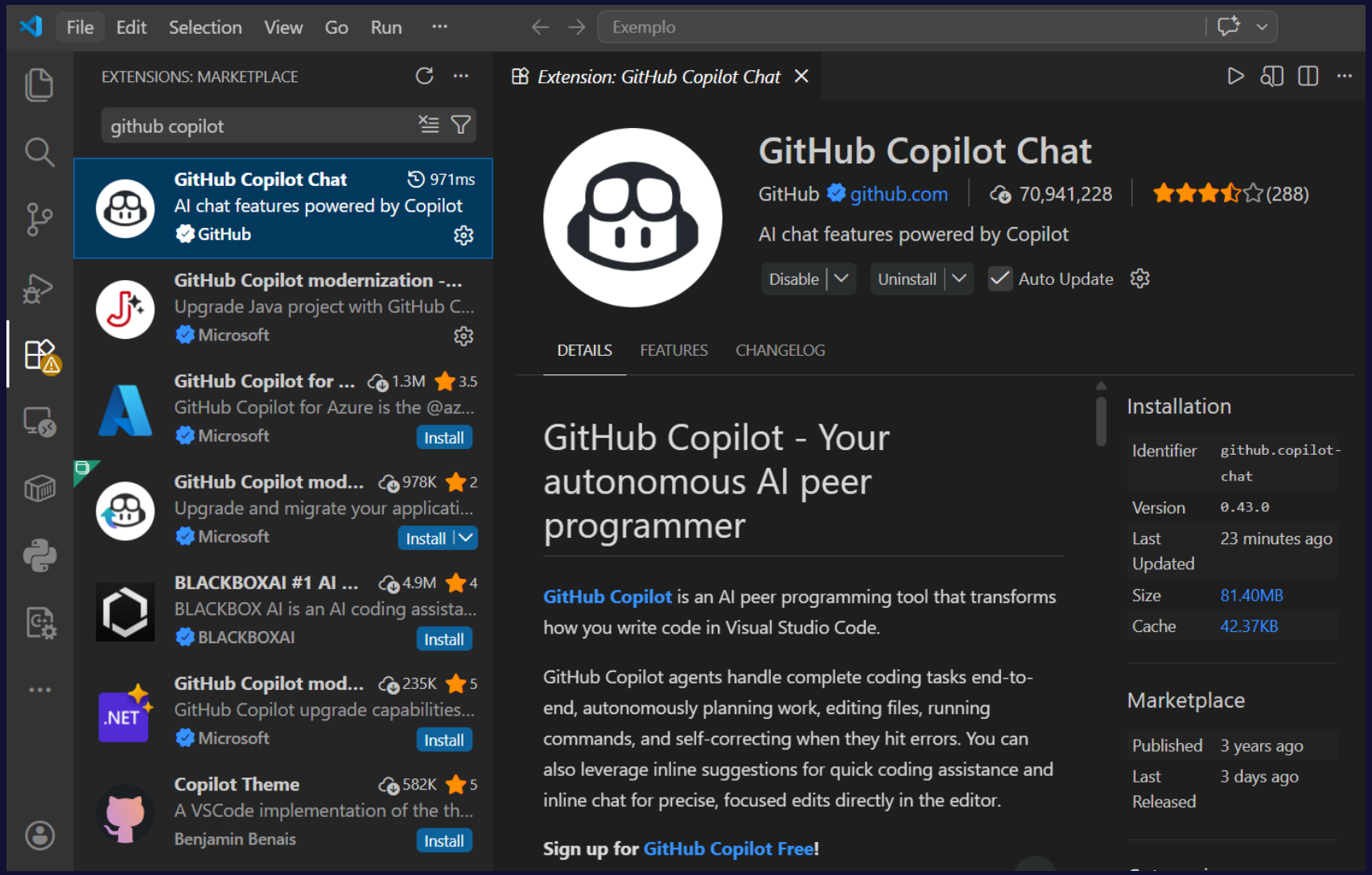

Figura 1: Buscando a extensão

Passo 2: Fazer login na sua conta GitHub

1. Após instalar, clique no ícone do Copilot na barra lateral esquerda (ou no canto inferior direito, onde aparece uma mensagem pedindo login)
2. Escolha **Sign in with GitHub**
3. O navegador abrirá uma página da GitHub – autorize o VS Code
4. Volte ao VS Code – a autenticação é concluída automaticamente

Atenção

Você precisa ter uma assinatura ativa do GitHub Copilot (paga, gratuita para estudantes/professores via GitHub Education, ou trial de 30 dias).

Passo 3: Verificar se o Copilot está funcionando

- Abra um arquivo de código (ex.: *app.py* ou *index.js*)

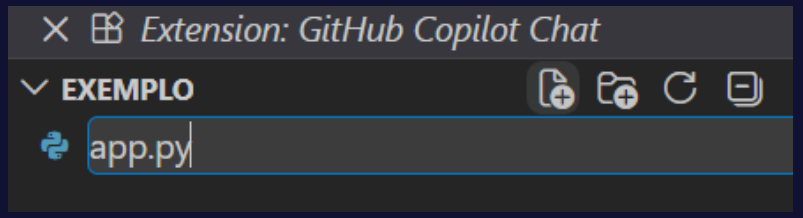

Figura 2: Criar arquivo

- Comece a digitar um comentário como: # função que calcula o fatorial de um número

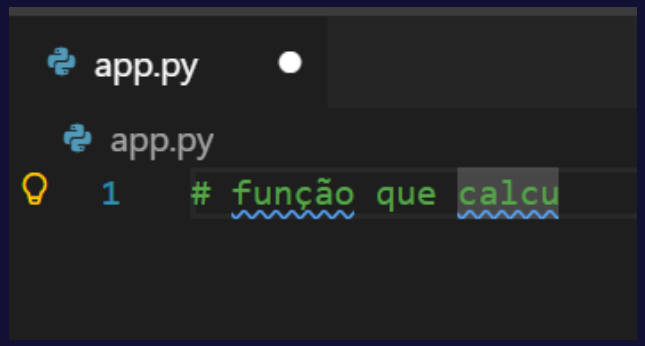

Figura 3: Fatorial - Autocomplete

Passo 3: Verificar se o Copilot está funcionando

- O Copilot deve sugerir automaticamente o código em cinza – pressione *Tab* para aceitar.

```
1   # função que calcula o fatorial de um número
2   def fatorial(n):
        if n == 0 or n == 1:
            return 1
        else:
            return n * fatorial(n - 1)
```

Figura 4: Sugestão

```
app.py > fatorial
1   # função que calcula o fatorial de um número
2   def fatorial(n):
3       if n == 0 or n == 1:
4           return 1
5       else:
6           return n * fatorial(n - 1)
```

Figura 5: Código Gerado

Passo 3: Verificar se o Copilot está funcionando

- Gere e main com o comentário: # main do programa

```
8   # main do programa
9   if __name__ == "__main__":
10      numero = int(input("Digite um número para calcular o
        fatorial: "))
11      resultado = fatorial(numero)
12      print(f"O fatorial de {numero} é: {resultado}")
```

Figura 6: Main Fatorial

Passo 3: Verificar se o Copilot está funcionando

Trocando Output por Terminal

- antes de executar, aperte ctrl+,
- digite nas configurações "code-runner: run"
- selecione a opção "rodar no terminal (run in terminal)"

Passo 3: Verificar se o Copilot está funcionando

- execute (ctrl+alt+n)

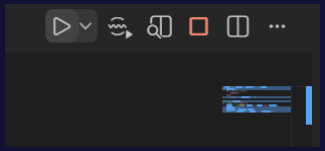

Figura 7: Executar

- coloque um número

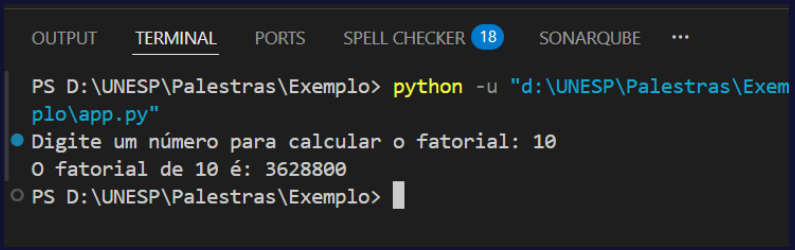

Figura 8: Fatorial de 10

Atalhos essenciais para usar o Copilot no VS Code

Atalho	Funcionalidade
Ctrl + I	**Inline Chat** – abre um chat diretamente na linha do código, ótimo para fazer perguntas ou pedir modificações localizadas.
Ctrl + Shift + I	**Chat lateral** – abre o painel de conversa completo, com contexto do projeto.
Tab	Aceitar a sugestão atual (completion).
Ctrl + Enter	Abrir o painel de sugestões alternativas (quando houver várias).

Atalhos essenciais para usar o Copilot no VS Code

DICA!

No VS Code, você pode personalizar esses atalhos em Arquivo > Preferências > Atalhos de teclado.

Exemplo rápido: Usando o inline chat (Ctrl+I)

- Em um arquivo Python vazio, pressione *Ctrl+I*
- Digite: *crie uma lista com os números de 1 a 10 e exiba apenas os pares*

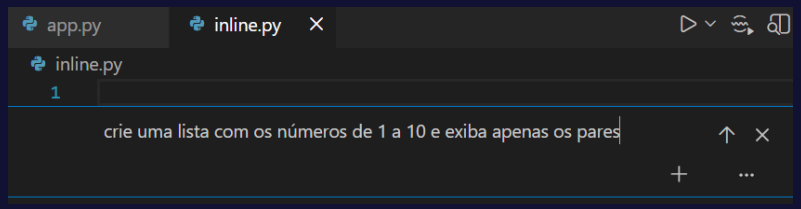

Figura 9: Código Inline

- O Copilot gera o código diretamente no editor – aceite com *Tab*

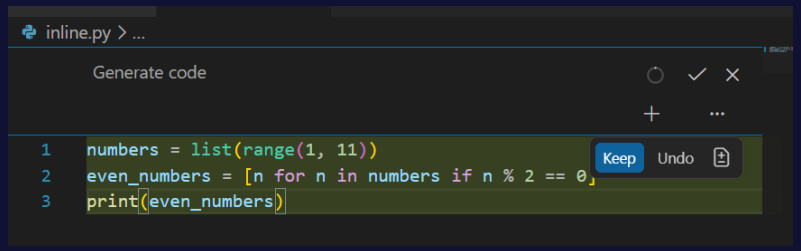

Figura 10: Código Gerado

Exemplo rápido: Usando o inline chat (Ctrl+I)

- Execute o código para ver o resultado

```
PS D:\UNESP\Palestras\Exemplo> python -u "d:\UNESP\Palestras\Exem
plo\inline.py"
[2, 4, 6, 8, 10]
PS D:\UNESP\Palestras\Exemplo>
```

Figura 11: Resultado Inline

Próximos passos (após a instalação)

- Explore o **Chat lateral** (*Ctrl+Shift+I*) – faça perguntas sobre seu código.
- Use comandos como */explain*, */tests*, */fix* dentro do chat.

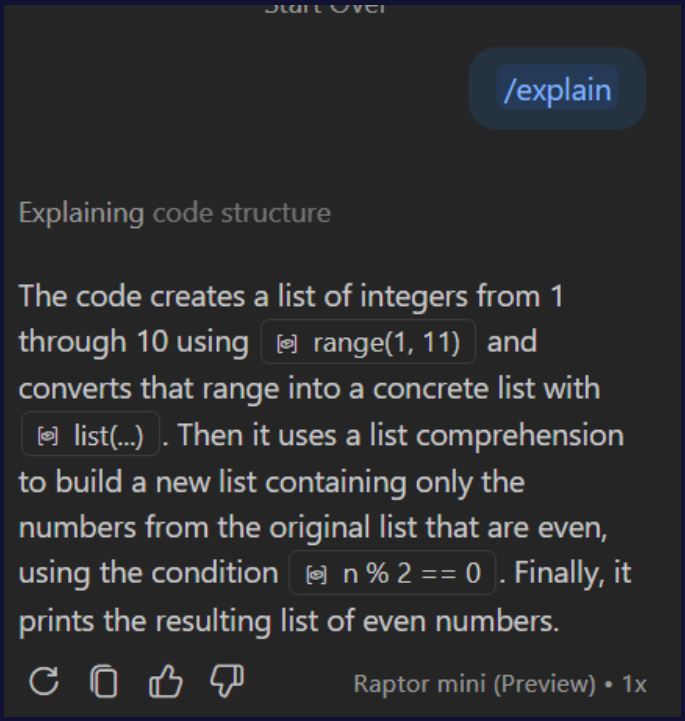

Figura 12: Explicação do Código

Testes

- executar /tests
- se quiser especificar pytest, fazer:
- /tests pytest

```
import pytest
from app import fatorial

def test_fatorial_zero():
    """Test fatorial for n=0, sho
    assert fatorial(0) == 1

def test_fatorial_one():
    """Test fatorial for n=1, sho
    assert fatorial(1) == 1

def test_fatorial_two():
    """Test fatorial for n=2, sho
    assert fatorial(2) == 2
```

Testes

- criar test_app.py na raiz
- copiar código
- definir pytest como ambiente (Ctrl+Shift+P)
- ir na aba testes (tubo de ensaio)
- rodar testes

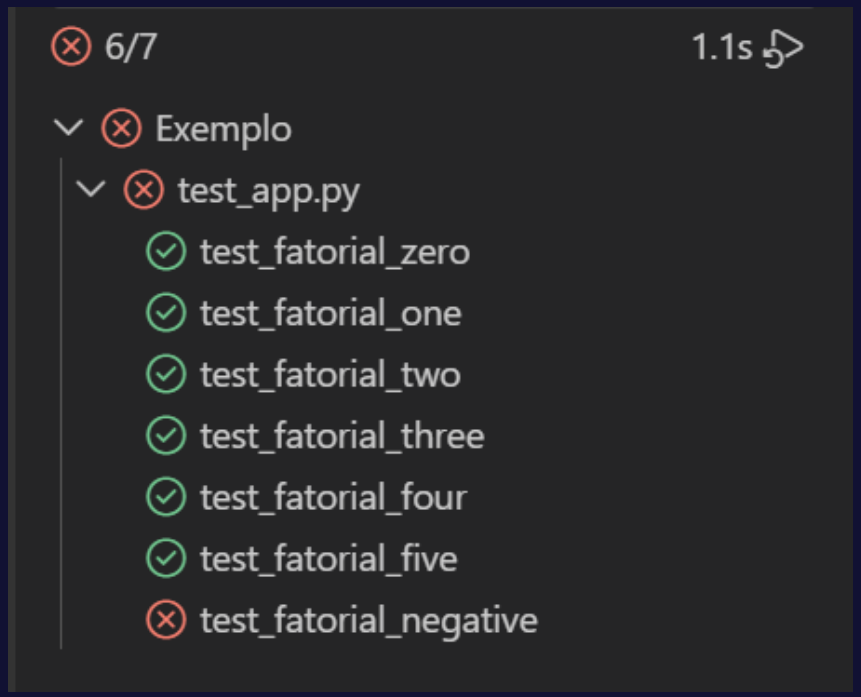

Figura 14: Um teste está errado!

Testes

- rodar /fix
- ele usa o código e os testes de base para identificar o erro e arrumar

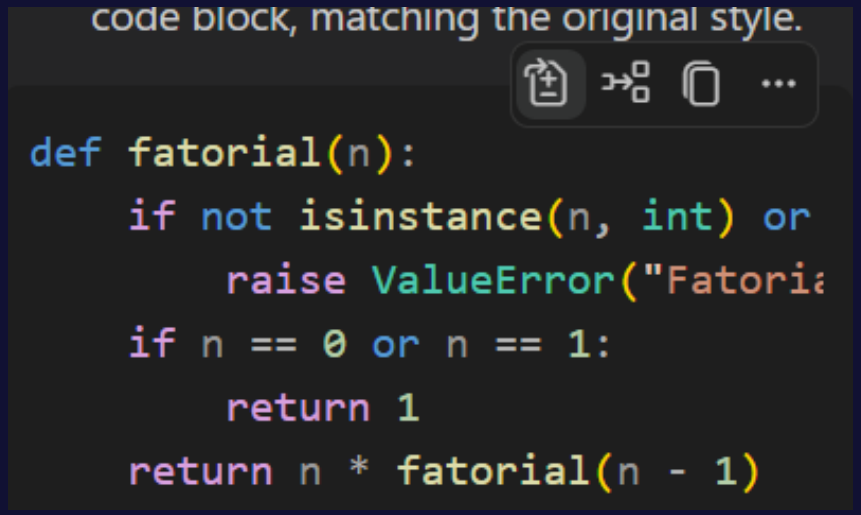

Figura 15: Aplicar solução no editor

Próximos passos (após a instalação)

- Ative o **Agent Mode** (se disponível na sua versão) para tarefas mais complexas.

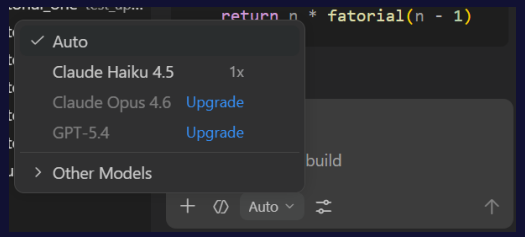

Figura 16: Modelos

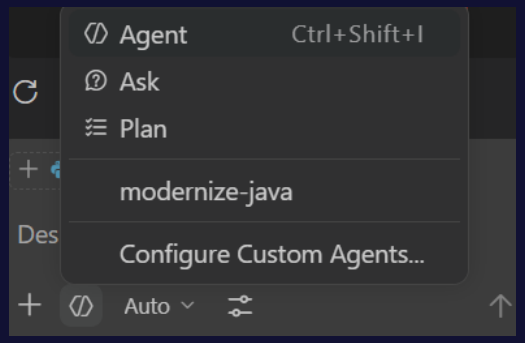

Figura 17: Modo Agente

Próximos passos (após a instalação)

- Consulte a documentação oficial: *https://docs.github.com/pt/copilot*

Agora você está pronto para codar com seu par de IA!

Section 3

Wordle Pokémon com GitHub Copilot

Demo: Wordle Pokémon com GitHub Copilot

Vamos construir um jogo interativo onde você adivinha o Pokémon escondido!

Temática: O jogo dá dicas sobre:

- **Tipo** (Fogo, Água, Planta...)
- **Habilidade** (Choque do Trovão, Chama...)
- **Ataques** (Soco, Mordida...)
- **Cor predominante** (Azul, Vermelho...)

O Copilot nos ajudará em cada etapa: completamento, chat, inline, refatoração e testes.

Passo 1: Gerar o esqueleto do jogo com completamento

1. No VS Code, crie *wordle_pokemon.py*
2. Digite o comentário:

```
# Jogo Wordle Pokémon:
# - Escolhe um Pokémon aleatório de uma lista.
# - Dá 5 dicas (tipo, habilidade, ataques, cor, geração).
# - O jogador tem 6 tentativas para adivinhar o nome.
```

3. O Copilot **sugere o código completo** – pressione *Tab* para aceitar.

Observação: A sugestão inclui uma lista de Pokémon com dicionários e a lógica principal.

Passo 2: Testar o jogo e corrigir com inline chat

- Execute o script (*python wordle_pokemon.py*).
- Você provavelmente encontrará erros.
- Selecione a linha onde tem o erro e aperte *Ctrl+I* (inline chat).
- Digite: *"Modifique para -resolver bug aqui-."*
- O Copilot sugere a correção – aceite com *Tab*.

Erro de tamanho de array

```
  File "d:\UNESP\Palestras\Exemplo\wordle_pokemon.py", line 94, i
n <module>
    jogar()
    ~~~~~^^
  File "d:\UNESP\Palestras\Exemplo\wordle_pokemon.py", line 81, i
n jogar
    letras_resultado = comparar_letras(pokemon_correto, entrada)
  File "d:\UNESP\Palestras\Exemplo\wordle_pokemon.py", line 39, i
n comparar_letras
    if letra == palavra_correta[i]:
                ~~~~~~~~~~~~~~~^^^
IndexError: string index out of range
(.venv) PS D:\UNESP\Palestras\Exemplo>
```

Figura 18: Erro ao percorrer vetor

```
📍 Tentativas restantes: 6
Digite seu palpite (ou 'dica'): pikachu
Entrada inválida: use 9 letras.

📍 Tentativas restantes: 6
Digite seu palpite (ou 'dica'): dragonite

❌ Errado! Veja as letras:
cinza cinza verde cinza amarelo cinza verde amarelo verde
```

Figura 19: Feedback Ruim

Passo 3: Adicionar dicas mais elaboradas com chat lateral

1. Abra o chat lateral (*Ctrl+Shift+I*).
2. Pergunte: *"Como posso gerar dicas variadas para cada Pokémon (ex: 'Este Pokémon é do tipo Fogo' em vez de apenas mostrar o dicionário?)"*
3. O Copilot sugere uma função *gerar_dica(pokemon, nivel)*.
4. Peça: *"Insira essa função no meu código, substituindo a exibição antiga."*
5. O Copilot mostra a diferença – aceite a refatoração.

Resultado: A cada tentativa, o jogador recebe uma dica nova e mais específica.

Passo 4: Refatorar com ajuda do Copilot

- O código ficou funcional, mas com repetições.
- No chat lateral, digite: *"Refatore o código para separar a lógica em funções: escolher_pokemon(), mostrar_dicas(), pedir_palpite(), verificar_vitoria()."*
- O Copilot gera o código refatorado e explica as mudanças.
- Use **Agent Mode** (se disponível) para aplicar automaticamente – ou copie/cole.

Vantagem: Código mais limpo, testável e reutilizável.

Passo 5: Gerar testes unitários automaticamente

1. No chat lateral, use o comando */tests* ou peça:
   "Gere testes com pytest para a função verificar_vitoria(palpite, pokemon_correto).
   Inclua casos: acerto, erro, maiúsculas/minúsculas."
2. O Copilot cria o arquivo *test_wordle_pokemon.py* com vários testes.
3. Execute *pytest test_wordle_pokemon.py* – os testes passam?.

Passo 5: Gerar testes unitários automaticamente

```
test_wordle_pokemon.py
  test_comparar_letras_retorna_cores_cor...
  test_comparar_letras_com_tamanho_in...
  test_mostrar_acertos_exibe_apenas_letr...
  test_validar_entrada_retorna_mensage...
  test_validar_entrada_retorna_mensage...
  test_validar_entrada_retorna_none_par...
  test_mostrar_dicas_retorna_texto_form...
  test_mostrar_dicas_sem_mais_dicas
  test_processar_dica_incrementa_e_impr...
  test_processar_dica_nao_excede_limite
  test_processar_palpite_acerto_retorna_t...
  test_processar_palpite_errado_reduz_te...
  test_escolher_pokemon_usa_random_c...

14  def test_comparar_letras_retorna_cores_corretas():

OUTPUT   TEST RESULTS   TERMINAL   PORTS   SPELL CHECKER 247   ...

wordle_pokemon.py:64: ValueError
---------------------------- Captured stdout call ------------
----------------

❌ Errado! Veja as letras:
========================== short test summary info ==========
=================
FAILED test_wordle_pokemon.py::test_comparar_letras_retorna_co
res_corretas - ...
FAILED test_wordle_pokemon.py::test_processar_palpite_errado_r
educ_tentativas
======================== 2 failed, 11 passed in 0.07s =======
=================
Finished running tests!
```

Figura 20: Testes do Wordle Pokémon

Passo 6: Melhorar a interface com inline chat

- O jogo no terminal está funcional, mas feio.
- Selecione a função que imprime as dicas e pressione `Ctrl+I`.
- Digite: *"Adicione cores usando ANSI escape codes (verde para acertos, vermelho para erros)."*
- O Copilot insere os códigos de cor – o terminal agora fica colorido.

Dica: O Copilot também sugere bibliotecas como `colorama` se necessário.

Passo 7: Adicionar um placar (gamificação)

- No chat lateral: *"Adicione um placar que conta quantas partidas o jogador venceu, quantas perdeu e mantém a pontuação máxima."*
- O Copilot gera o código para um placar persistente (usando JSON ou variáveis globais).
- Peça para integrar no loop principal – o Copilot mostra exatamente onde colocar.

Desafio para casa

- Adicione mais Pokémon à base de dados.
- Crie um modo "difícil"com menos dicas.
- Use o Copilot para gerar uma interface web com Streamlit.

Agora é sua vez de experimentar o GitHub Copilot no VS Code!

Section 4

Considerações finais

Limitações do GitHub Copilot – o que ele NÃO faz

- **Não substitui o raciocínio humano:** Pode gerar código incorreto, ineficiente ou vulnerável.
- **Contexto limitado:** Nem sempre entende o projeto inteiro (especialmente códigos muito grandes ou com arquitetura complexa).
- **Pode gerar código com viés ou bugs:** Treinado em código público – repete erros comuns.
- **Depende de internet:** Precisa de conexão ativa para a maioria das funcionalidades.
- **Custo:** Assinatura pode ser paga.

Use o Copilot como um assistente, não como um substituto da sua expertise.

Boas práticas para usar o Copilot

1. Revise sempre o código sugerido
2. Seja específico nos comentários e prompts
3. Use o chat para aprender, não apenas para gerar código
4. Mantenha seu código limpo e organizado
5. Não confie cegamente em testes gerados pelo /tests
6. Atualize a extensão regularmente

Lembre-se: você é o líder, o Copilot é o parceiro.

Onde aprender mais sobre GitHub Copilot

- **Documentação oficial:** *https://docs.github.com/pt/copilot*
- **Cursos gratuitos:**
  - GitHub Skills – *https://github.com/skills* (busque por "Copilot")
  - Microsoft Learn – *https://learn.microsoft.com/pt-br/training/browse/?terms=copilot*
- **Comunidade:**
  - GitHub Copilot Discussion Board
  - VS Code YouTube Channel (demos semanais)
- **Repositórios de exemplo:** Pesquise por "copilot-demos"no GitHub.

Prática é o melhor caminho – experimente em seus próprios projetos!

Perguntas e respostas (Q&A)

? ? ?

Importante!

Programar com IA não é sobre substituir pessoas, mas sobre ampliar suas capacidades e acelerar o aprendizado.

Estou à disposição para conversarmos após a sessão também.

Obrigado!

Leonardo Tórtoro Pereira
leonardo.pereira@unesp.br
Instagram pessoal: @leonardotphd
profissional: @prof.leo.pereira
Grupo de pesquisa: Re-G3X